

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1510

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I **Soundariya M(192524342)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Soundariya M**

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

❖ HDFS-Based Storage Solution for a Media Platform

Main Sentence

An HDFS-based storage solution using three-way replication, Name Node high availability, and cost-efficient storage techniques can provide reliable and scalable storage for a media platform.

1. Data Storage Using HDFS

A media platform generates a large amount of videos, images, audio files, and other media content. **Hadoop Distributed File System (HDFS)** can be used to store this large volume of data. HDFS divides large files into blocks and distributes these blocks across multiple Data Nodes. This allows the system to store huge datasets and process them efficiently.

2. Replication Factor

The replication factor should be fixed at **3**, because the given constraint states that it cannot exceed 3. Each important data block will have three copies stored on different Data Nodes. The replicas should preferably be placed on different racks. If one Data Node fails, another copy can be used without losing access to the data.

3. High Availability

To maintain data availability above **99.9%**, Name Node High Availability can be implemented. An Active Name Node and a Standby Name Node can be used. If the Active Name Node fails, the Standby Name Node can take over. This prevents the Name Node from becoming a single point of failure.

4. Cost Efficiency

Storage cost can be reduced by using **data compression and storage tiering**. Frequently accessed media files can be stored on high-performance storage, while old or rarely accessed files can be moved to low-cost storage. Unnecessary duplication should also be avoided.

5. Fault Tolerance

The three replicas provide protection against Data Node failures. Rack-aware replica placement also protects the system if an entire rack becomes unavailable. Thus, the media files remain accessible even during hardware failures.

Conclusion

Therefore, HDFS with a replication factor of 3, Name Node High Availability, rack-aware storage, compression, and storage tiering can satisfy the requirements of **99.9% availability, maximum three replicas, and cost-efficient storage**.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

❖ MapReduce Workflow Within a 20-Minute Batch Window

Main Sentence

A MapReduce workflow using parallel processing, data locality, a Combiner, and optimized task distribution can process large log data within the required 20-minute batch window.

1. Log Data Storage

The log data is first stored in **HDFS**. The log files are divided into blocks, and these blocks are distributed across the available Data Nodes. Storing the data in HDFS allows multiple processing tasks to work on different parts of the data simultaneously.

2. Map Phase

In the Map phase, multiple Mapper tasks process different portions of the log data in parallel. The Mapper can extract information such as IP address, timestamp, URL, HTTP status, and user activity. It generates key-value pairs from the input data.

3. Combiner

A Combiner can be used after the Map phase to perform local aggregation. For example, if one Mapper finds that a particular URL appears 100 times, the Combiner can combine these values before sending them to the Reducer. This reduces the amount of data transferred through the network.

4. Shuffle and Sort

During the Shuffle and Sort phase, MapReduce groups all values belonging to the same key. The grouped data is then transferred to the appropriate Reducer. This step prepares the data for final processing.

5. Reduce Phase

Reducer tasks perform the final aggregation and generate the required results. For example, Reducers can calculate the total number of visits to each URL, total errors, or the number of requests received from each IP address.

6. Task Distribution

Since the cluster has limited nodes, the number of Mapper and Reducer tasks should be selected according to available CPU and memory. Multiple tasks can run simultaneously on each node. **Data locality** should also be used so that computation happens close to where the input data is stored.

7. Meeting the 20-Minute Constraint

Compression, data locality, parallel Map tasks, a Combiner, and optimized Reducer allocation can reduce network communication and processing time. These techniques help the workflow complete within the required **20-minute batch window**.

Conclusion

Thus, an optimized MapReduce workflow with parallel task execution and efficient data transfer can process large log datasets within the given time constraint even with limited cluster nodes.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

❖ **YARN Scheduler Policy for ETL, Reporting, and Machine Learning**

Main Sentence

A priority-based YARN scheduling policy with fair resource allocation and automatic failure recovery can efficiently manage ETL, reporting, and machine learning workloads.

1. Job Classification

The YARN cluster contains three major types of jobs: **ETL, reporting, and machine learning**. These jobs have different resource requirements and priorities, so they should be placed into separate queues.

2. ETL Priority

ETL jobs should receive the **highest priority** because they prepare and transform data that may be required by reporting and machine learning applications. A larger resource capacity can therefore be allocated to the ETL queue.

3. Reporting Jobs

Reporting jobs can be given **medium priority**. They require sufficient resources to generate reports within an acceptable time, but they should not completely block high-priority ETL processing.

4. Machine Learning Jobs

Machine learning jobs can receive normal priority. ML jobs may require large CPU and memory resources and may run for a long time. Therefore, resource limits should be applied so that ML jobs do not consume all cluster resources.

5. Fairness

Although ETL has higher priority, reporting and machine learning jobs should still receive their minimum guaranteed resources. Fair scheduling within each queue can prevent one job from using all the available resources.

6. Node Failure Recovery

YARN can detect when a worker node fails. Containers running on the failed node can be restarted on healthy nodes. The Application Master manages the execution of the application and helps coordinate recovery.

7. Scheduling Policy

A **Capacity Scheduler or priority-based scheduler** can be used. The scheduler can allocate resources according to priority while maintaining minimum resource availability for other workloads.

Conclusion

Therefore, the proposed YARN policy provides **high priority to ETL, fair resource sharing for reporting and ML, and automatic recovery from node failures**, satisfying all the given constraints.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

❖ Enterprise Data Ingestion from SAN, NAS, and Operational Databases

Main Sentence

A centralized data ingestion architecture using Apache Ni Fi, incremental loading, HDFS, and reliable backup storage can integrate SAN, NAS, and operational databases while reducing data duplication.

1. Data Sources

The enterprise receives data from different sources such as **SAN, NAS, and operational databases**. These systems store files, documents, transactions, and other business information in different formats.

2. Apache Ni Fi as Ingestion Layer

Apache Ni Fi can be used as the central data ingestion tool. It collects data from the different sources and transfers it to the big data platform. Ni Fi also supports data routing, transformation, validation, monitoring, and error handling.

3. SAN and NAS Integration

Files from SAN and NAS can be transferred into the Hadoop platform using NiFi. Instead of copying all files repeatedly, the system can identify newly created or modified files and transfer only those files. This reduces unnecessary data movement.

4. Operational Database Integration

Operational databases can be connected using database connectors. Instead of copying the complete database every time, **incremental extraction** can be performed using timestamps, transaction IDs, or change tracking. This reduces processing time and network usage.

5. Minimizing Duplication

Duplicate files and records can be identified using checksums, unique record IDs, metadata, and incremental ingestion. Before storing the data, the system can check whether the same data already exists.

6. HDFS Storage

After ingestion and validation, the data can be stored in HDFS. The data can be organized using partitions such as source, date, department, or data type. Compression can also be used to reduce storage requirements.

7. Backup Reliability

Important data should be backed up to separate storage. Regular backup schedules can be created according to the importance of the data. HDFS replication can also provide protection against individual Data Node failures

.

Conclusion

Therefore, the proposed architecture provides **centralized ingestion, reliable backup, scalable storage, and minimum duplication** while integrating SAN, NAS, and operational databases.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

❖ End-to-End Hadoop Ecosystem Solution

Main Sentence

An end-to-end Hadoop ecosystem using Apache NiFi, HDFS, YARN, Hive, Spark, and security mechanisms can provide secure data movement, high availability, and efficient ETL integration.

1. Data Sources

The system can collect data from different sources such as operational databases, files, APIs, applications, sensors, and other enterprise systems. These sources may generate both structured and unstructured data.

2. Apache NiFi for Data Ingestion

Apache NiFi acts as the **data ingestion layer**. It collects data from different sources and moves it into the Hadoop environment. NiFi can perform data collection, validation, filtering, transformation, routing, and error handling.

3. Secure Data Movement

Security is important when transferring sensitive enterprise data. **TLS/SSL** can be used to encrypt communication between systems. **Kerberos** can provide authentication for Hadoop services and users. Authorization and access-control policies can ensure that only permitted users can access sensitive data.

4. HDFS for Storage

The processed data can be stored in HDFS. HDFS provides distributed and scalable storage by dividing files into blocks and storing them across multiple DataNodes. A replication factor of 3 can be used to improve fault tolerance.

5. High Availability

High availability can be provided using **Active and Standby NameNodes**. If the Active NameNode fails, the Standby NameNode can take over. Multiple DataNodes and HDFS replication also protect against hardware failures.

6. YARN for Resource Management

YARN manages the computing resources of the Hadoop cluster. It allocates CPU and memory to different applications and allows multiple workloads to run simultaneously. YARN can manage ETL, reporting, analytics, and machine learning workloads.

7. Hive for Data Analysis

Apache Hive can be used for SQL-based analysis and reporting. It allows users to query large datasets stored in HDFS using SQL-like commands without writing complex MapReduce programs.

8. Spark for Fast Processing

Apache Spark can be integrated with the Hadoop ecosystem for fast data processing and machine learning. Spark can process large datasets efficiently and can support advanced analytics and ML workloads.

9. ETL Pipeline Integration

The complete ETL flow can be designed as:

Data Sources → NiFi → Validation → Transformation → HDFS → YARN → Hive/Spark → Analytics

NiFi manages the movement and transformation of data, while Hadoop components provide storage, resource management, and processing.

10. Constraint Satisfaction

The **secure data movement** requirement is satisfied using TLS/SSL, Kerberos, authentication, and authorization. The **high availability** requirement is satisfied using NameNode HA, multiple DataNodes, HDFS replication, and YARN availability mechanisms. The **NiFi/ETL integration** requirement is satisfied because NiFi collects, transforms, routes, and transfers data into HDFS and can connect the ingestion pipeline with Hadoop processing tools.

Conclusion

Therefore, the proposed end-to-end Hadoop ecosystem provides a **secure, highly available, fault-tolerant, scalable, and efficient big data platform**. It integrates Apache NiFi with HDFS, YARN, Hive, and Spark to support complete data ingestion, storage, processing, and analytics.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured